

COMPILADORES

PROJETO Fase 1: Etapa 1

Expressões Regulares e Gramática Livre de Contexto

Integrante	RA
Caio Ariel Cardoso Saraiva	10439611
Isabela Hissa Pinto	10441873
Kaique Barros Paiva	10441787

Este documento apresenta as expressões regulares dos tokens e a gramática livre de contexto da linguagem MiniVisualg, subconjunto do Visualg definido pelos exemplos do Anexo I do enunciado. A gramática foi escrita pensando no analisador sintático descendente da Etapa 3.

1. Alfabeto

O código fonte é formado pelos seguintes símbolos:

- letras: a . . z e A . . Z
- dígitos: 0 . . 9
- sublinhado: _
- símbolos: + - * / \ < > = () [] , : . "
- espaços em branco: espaço, tabulação, \r e \n (quebra de linha)
- dentro de cadeias (entre aspas) e comentários qualquer caractere é aceito, inclusive acentos

Qualquer caractere fora desse alfabeto (por exemplo @, #, \$, {) fora de uma cadeia ou comentário gera **ERRO LÉXICO**.

2. Definições regulares auxiliares

```
letra  = [a-zA-Z]
digito = [0-9]
branco = ( " " | \t | \r | \n )+
```

3. Expressões regulares dos tokens

Token	Expressão regular	Exemplo
ID	(letra _)(letra dígito _)*	idade, eh_par
NUM_INT	dígito+	42
NUM_FLOAT	dígito+ "." dígito+	1.60
STRING	" [^\n]* "	"Olá, mundo!"
OP_REL	< <= = > >= <>	>=
ATRIB	<-	<-
SOMA	+	
SUB	-	
MULT	*	
DIV	/	
DIV_INT	\	
ABRE_PAR / FECHA_PAR	(/)	
ABRE_COL / FECHA_COL	[/]	
VIRGULA	,	
DOIS_PONTOS	:	
PONTO_PONTO	..	1..4
palavra reservada	mesma ER de ID, confirmada na tabela da seção 4	se

Padrões reconhecidos e descartados (não geram token):

Padrão	Expressão regular
comentário	// [^\n]*
espaço em branco	branco

Atributos de cada token

Token	Atributo guardado na struct
ID	table_index (posição na tabela de símbolos)
STRING	table_index (o texto fica na tabela de símbolos)
NUM_INT	int_value
NUM_FLOAT	float_value
OP_REL	op_code (OP_LT, OP_LE, OP_EQ, OP_GT, OP_GE, OP_NE)
demais	sem atributo

4. Palavras reservadas

algoritmo	var	inicio	fimalgoritmo
inteiro	real	caractere	logico
vetor	de		
se	entao	senao	fimse
para	ate	passo	faca
enquanto	fimenquanto		fimpara
procedimento	fimprocedimento		
funcao	fimfuncao	retorne	
leia	escreva	escreval	
verdadeiro	falso		
e	ou	mod	

Todo lexema que casa com a ER de ID é procurado nessa tabela. Se estiver nela, vira o token da palavra reservada, senão vira ID. A comparação não diferencia maiúsculas de minúsculas, porque o anexo usa MOD e E em maiúsculo.

5. Regras de desambiguação do léxico

- **Maior casamento (longest match):** o léxico sempre pega o maior lexema possível. Assim <= vira um OP_REL só e não < seguido de =, e <- vira ATRIB.
- **O < tem quatro saídas:** se o próximo caractere for = é OP_LE, se for > é OP_NE, se for - é ATRIB, senão é OP_LT.
- **Número seguido de . . .:** em 1 . . 4, depois de ler 1 e ver um ponto, o léxico olha o caractere seguinte. Só é NUM_FLOAT se depois do ponto vier um dígito. Se vier outro ponto, ele devolve NUM_INT e o . . vira PONTO_PONTO.
- **Sinal negativo:** números não têm sinal na ER. Em passo -2 o - é o token SUB e a gramática trata como menos unário.
- **Separação por espaço:** apesar do enunciado dizer que os lexemas estão separados por espaço, os exemplos do anexo não seguem isso (escreval(" . . ."), nomes[1]). O léxico reconhece os tokens mesmo sem espaço entre eles.

Erros léxicos

- caractere fora do alfabeto (ex: @)
- cadeia aberta que chega no fim da linha ou do arquivo sem fechar aspas
- número real mal formado, como 1. sem dígito depois do ponto

6. Gramática livre de contexto

Notação EBNF: { X } é zero ou mais repetições de X, [X] é X opcional e | separa alternativas. Não-terminais em minúsculo, terminais (tokens) em MAIÚSCULO ou entre aspas. O símbolo inicial é programa.

6.1 Estrutura do programa

```
programa      -> ALGORITMO STRING declaracoes INICIO comandos FIMALGORITMO

declaracoes   -> { secao_var | procedimento | funcao }

secao_var     -> VAR { decl_var }
decl_var      -> lista_ids ":" tipo
lista_ids     -> ID { ",", ID }

tipo          -> tipo_basico
               | VETOR "[" NUM_INT ".." NUM_INT "]" DE tipo_basico
tipo_basico   -> INTEIRO | REAL | CARACTERE | LOGICO
```

declaracoes aceita var, procedimentos e funções em qualquer ordem porque o anexo mostra as duas formas (função antes do var e programa sem seção var).

6.2 Sub-rotinas

```
procedimento  -> PROCEDIMENTO ID [ "(" parametros ")" ]
               INICIO comandos FIMPROCEDIMENTO

funcao        -> FUNCAO ID "(" parametros ")" ":" tipo_basico
               INICIO comandos FIMFUNCAO

parametros    -> parametro { ",", parametro }
parametro     -> ID ":" tipo_basico
```

6.3 Comandos

```
comandos      -> { comando }

comando       -> cmd_id
               | cmd_se
               | cmd_para
               | cmd_enquanto
               | cmd_leia
               | cmd_escreva
               | cmd_retorne

cmd_id        -> ID resto_id
resto_id      -> "<-" expr
               | "[" expr "]" "<-" expr
               | "(" argumentos ")"
               | ε

cmd_se        -> SE expr ENTAO comandos [ SENAO comandos ] FIMSE
cmd_para      -> PARA ID DE expr ATE expr [ PASSO expr ] FACA comandos FIMPARA
cmd_enquanto  -> ENQUANTO expr FACA comandos FIMENQUANTO
cmd_leia      -> LEIA "(" variavel ")"
cmd_escreva   -> ( ESCREVA | ESCREVAL ) "(" argumentos ")"
cmd_retorne   -> RETORNE expr

variavel      -> ID [ "[" expr "]" ]
argumentos    -> expr { ",", expr }
```

6.4 Expressões

Cada nível corresponde a um nível de precedência, do menor para o maior.

```
expr          -> expr_e { OU expr_e }
expr_e        -> expr_rel { E expr_rel }
expr_rel      -> expr_arit [ OP_REL expr_arit ]
expr_arit     -> termo { ( "+" | "-" ) termo }
termo         -> fator { ( "*" | "/" | "\" | MOD ) fator }

fator         -> NUM_INT
               | NUM_FLOAT
               | STRING
               | VERDADEIRO
               | FALSO
               | "-" fator
               | "(" expr ")"
               | ID [ "[" expr "]" | "(" argumentos ")" ]
```

Precedência	Operadores
1 (menor)	ou
2	e
3	= <> < <= > >=
4	+ -
5	* / \ mod
6 (maior)	- unário, parênteses

Todos os operadores binários são associativos à esquerda, já que a repetição { } é processada da esquerda para a direita. O relacional aparece no máximo uma vez por nível ([]), então a $a < b < c$ sem parênteses é erro sintático.

7. Por que a gramática serve para análise descendente

O analisador da Etapa 3 é descendente recursivo, então a gramática foi escrita para ser LL(1):

- **Sem recursão à esquerda.** Em vez de `expr_arit -> expr_arit + termo`, usamos `expr_arit -> termo { + termo }`. A recursão à esquerda faria a função chamar a si mesma para sempre.
- **Fatorada à esquerda.** Atribuição simples, atribuição em vetor e chamada de procedimento começam com ID. O ID foi colocado em evidência em `cmd_id` e a escolha é feita pelo token seguinte em `resto_id`.
- **Sem else pendurado.** Todo se termina com `fimse`, então cada `senao` pertence sem dúvida ao se mais próximo que ainda está aberto.
- **Cada escolha é decidida pelo token atual (lookahead)**, como mostra a tabela abaixo.

Não-terminal	Token atual	Alternativa escolhida
declaracoes	VAR / PROCEDIMENTO / FUNCAO	secao_var / procedimento / funcao
declaracoes	INICIO	termina a repetição
comando	ID	cmd_id
comando	SE / PARA / ENQUANTO	cmd_se / cmd_para / cmd_enquanto
comando	LEIA / ESCREVA / ESCREVAL / RETORNE	comando correspondente
comandos	FIMALGORITMO, FIMSE, SENAO, FIMPARA, FIMENQUANTO, FIMPROCEDIMENTO, FIMFUNCAO	termina a repetição
resto_id	<- / [/ (	atribuição / atribuição em vetor / chamada
resto_id	qualquer outro	ϵ (chamada de procedimento sem parâmetros)

Nenhum comando começa com (ou [, por isso o ϵ de resto_id não conflita com o próximo comando.

8. Equivalência em BNF

Se for exigida BNF pura, cada repetição { X } vira uma regra recursiva à direita com ϵ , e cada [X] vira uma alternativa com ϵ . Exemplos:

```
lista_ids      -> ID resto_ids
resto_ids      -> "," ID resto_ids |  $\epsilon$ 

expr_arit      -> termo expr_arit'
expr_arit'     -> "+" termo expr_arit' | "-" termo expr_arit' |  $\epsilon$ 

cmd_se         -> SE expr ENTAO comandos parte_senao FIMSE
parte_senao    -> SENAO comandos |  $\epsilon$ 
```

9. Exemplo de derivação

Programa:

```
algoritmo "PrimeiroPasso"
var
inicio
escreval("Olá, mundo!")
fimalgoritmo
```

Tokens: ALGORITMO STRING VAR INICIO ESCREVAL (STRING) FIMALGORITMO

Derivação mais à esquerda:

```
programa
=> ALGORITMO STRING declaracoes INICIO comandos FIMALGORITMO
=> ALGORITMO STRING secao_var INICIO comandos FIMALGORITMO
=> ALGORITMO STRING VAR INICIO comandos FIMALGORITMO
=> ALGORITMO STRING VAR INICIO comando FIMALGORITMO
=> ALGORITMO STRING VAR INICIO cmd_escreva FIMALGORITMO
=> ALGORITMO STRING VAR INICIO ESCREVAL ( argumentos ) FIMALGORITMO
=> ALGORITMO STRING VAR INICIO ESCREVAL ( expr ) FIMALGORITMO
   (expr => expr_e => expr_rel => expr_arit => termo => fator => STRING)
=> ALGORITMO STRING VAR INICIO ESCREVAL ( STRING ) FIMALGORITMO
```

10. Decisões e limitações

- Só estão na gramática as construções que aparecem no Anexo I. Não há `nao`, `repita`, `escolha`, matrizes nem variáveis locais dentro de sub-rotinas.
- Além dos tokens da Figura 2 do enunciado, foi criado um token para cada palavra reservada e para cada símbolo, e o operador `<>` ganhou o sub-código `OP_NE`, que não estava na figura. No Visualg a igualdade é `=` e não `==`.
- A gramática aceita retorne fora de função e não confere tipos. Essas checagens são semânticas e ficam fora desta fase.
- O exemplo "ProcedimentosComParametros" do anexo parece ter dois programas colados. Consideramos que as sub-rotinas são declaradas antes do início, como nos outros exemplos.